

План лекции.

- Функция WinMain.
- Сообщения Windows.
- Оконная функция.
- GDI. Меню в главном окне приложения.
- Модальный и немодальный диалоги.
- Таймеры Windows и служба времени.

Цели лекции.

Познакомиться с набором библиотек WinAPI. Получить первичные сведения о функциях WinAPI и их применением в различных ситуациях.

Теоретическая часть лекции.

Windows API (англ. *application programming interfaces*) — общее наименование целого набора базовых функций интерфейсов программирования приложений операционных систем семейств *Microsoft Windows* корпорации «Майкрософт» и совместимой с ними свободной бесплатной операционной системы *ReactOS*. Является самым прямым способом взаимодействия приложений с *Windows* и *ReactOS*. Для создания программ, использующих *Windows API*, «Майкрософт» выпускает комплект разработчика программного обеспечения, который называется *Platform SDK*, и содержит документацию, набор библиотек, утилит и других инструментальных средств для разработки.

Общие сведения

Windows API спроектирован для использования в языке Си для написания прикладных программ, предназначенных для работы под управлением операционной системы MS Windows. Работа через *Windows API* — это наиболее близкий к операционной системе способ взаимодействия с ней из прикладных программ. Более низкий уровень доступа, необходимый только для драйверов устройств, в текущих версиях *Windows* предоставляется через *Windows Driver Model*.

Windows API представляет собой множество функций, структур данных и числовых констант, следующих соглашениям языка Си. Все языки программирования, способные вызывать такие функции и оперировать такими типами данных в программах, исполняемых в среде Windows, могут пользоваться этим API. В частности, это языки C++, Pascal, Visual Basic и многие другие.

Для облегчения программирования под Windows, в компании Microsoft и сторонними разработчиками было предпринято множество попыток создать библиотеки и среды программирования, частично или полностью скрывающие от программиста особенности Windows API, и предоставляющие ту или иную часть его возможностей в более удобном виде. В частности, сама Microsoft в разное время предлагала библиотеки Active Template Library (ATL)/Windows Template Library (WTL), Microsoft Foundation Classes (MFC), .Net/WinForms/WPF. Компания Borland (ныне Embarcadero, её преемник в части средств разработки) предлагала OWL и VCL. Есть кросс-платформенные библиотеки, такие как Qt, Tk и многие другие. Весомая часть этих библиотек сконцентрирована на облегчении программирования графического интерфейса пользователя.

Для облегчения переноса на другие платформы программ, написанных с опорой на Windows API, сделана библиотека Wine.

Версии

- **Win16** — первая версия *WinAPI* для 16-разрядных версий *Windows*. Изначально назывался *Windows API*, позднее был ретроспективно переименован в *Win16* для отличия от *Win32*. Описан в стандарте *ECMA-234*.
- **Win32** — 32-разрядный *API* для современных версий *Windows*. Самая популярная ныне версия. Базовые функции реализованы в динамически подключаемых библиотеках `kernel32.dll` и `advapi32.dll`; базовые модули графического интерфейса пользователя — в `user32.dll` и `gdi32.dll`. *Win32* появился вместе с *Windows NT* и затем был перенесён в несколько ограниченном виде в системы серии *Windows 9x*. В современных версиях *Windows*, происходящих от *Windows NT*, работу *Win32 GUI* обеспечивают два модуля: `csrss.exe` (процесс исполнения клиент-сервер), работающий в пользовательском режиме, и `win32k.sys` в режиме ядра. Работу же системы обеспечивает ядро — `ntoskrnl.exe`.
- **Win32s** — подмножество *Win32*, устанавливаемое на семейство 16-разрядных систем *Windows 3.x* и реализующее ограниченный набор функций *Win32* для этих систем.
- **Win64** — 64-разрядная версия *Win32*, содержащая дополнительные функции *Windows* на платформах *x86-64* и *IA-64*.

Несколько слов об API, Win32, DLL

API (*Application Programming Interface* - интерфейс прикладных программ) - это множество функций, организованных, обычно, в виде *DLL*. Функции *API* позволяют организовать интерфейс между прикладной программой и средой, в которой работает эта программа. Вызов функций *API* позволяет программе получать доступ к ресурсам среды и управлять ее работой. Как правило, *API* задает стандарт взаимодействия среды и прикладной программы.

Win32 - это название интерфейса, ориентированного на 32-х разрядные приложения и реализованного на таких известных платформах как *Windows 95*, *Windows 98*, *Windows NT*, *Windows CE*. Функции, составляющие этот интерфейс, позволяют прикладной программе получать доступ к ресурсам операционной системы и управлять ее работой. Более ранние версии *Windows* используют интерфейс, известный как *Win16*. Конечно, не все функции, составляющие интерфейс *Win32*, реализованы в полной мере на всех платформах, так что вызов одной и той же функции под *NT* приведет к определенному результату, а под *Windows 95* работает как вызов заглушки. Любое из приложений, работающее в среде *Windows*, прямо или косвенно вызывает функции, входящие в *Win32 API*.

Функции, составляющие *Win32* интерфейс, организованы в виде нескольких динамически подключаемых библиотек (*DLL*) и исполняемых файлов. Говоря о *Win32 API*, следует в первую очередь упомянуть три основные библиотеки:

1. **Kernel32.dll**. Эта библиотека предназначена для работы с объектами ядра операционной системы и ее функции позволяют управлять памятью и другими системными ресурсами.
2. **User32.dll**. Здесь сосредоточены функции для управления окнами - основным видом объектов операционной системы. Обработка сообщений, работа с меню, таймерами, все это выполняют функции этой *DLL*.
3. **GDI32.dll**. Эта библиотека, обеспечивающая графический интерфейс операционной системы (*Graphics Device Interface*). Функции управления выводом на экран дисплея, управления выводом принтера, функции для работы со шрифтами - все они входят в состав этой библиотеки.

Заметьте, *Win API* функции находятся не только в этих библиотеках. С другой стороны *API* функции не обязательно входят в состав *Win32* интерфейса. Например, *MAPI* интерфейс (*Messaging Application Programming Interface*) составляют функции, предназначенные для обработки сообщений электронной почты, *TAPI* (*Telephone API*) - функции работы с телефонными

сообщениями. MAPI, TAPI, также как и Win32 это некоторый набор функций, задающий определенный стандарт взаимодействия

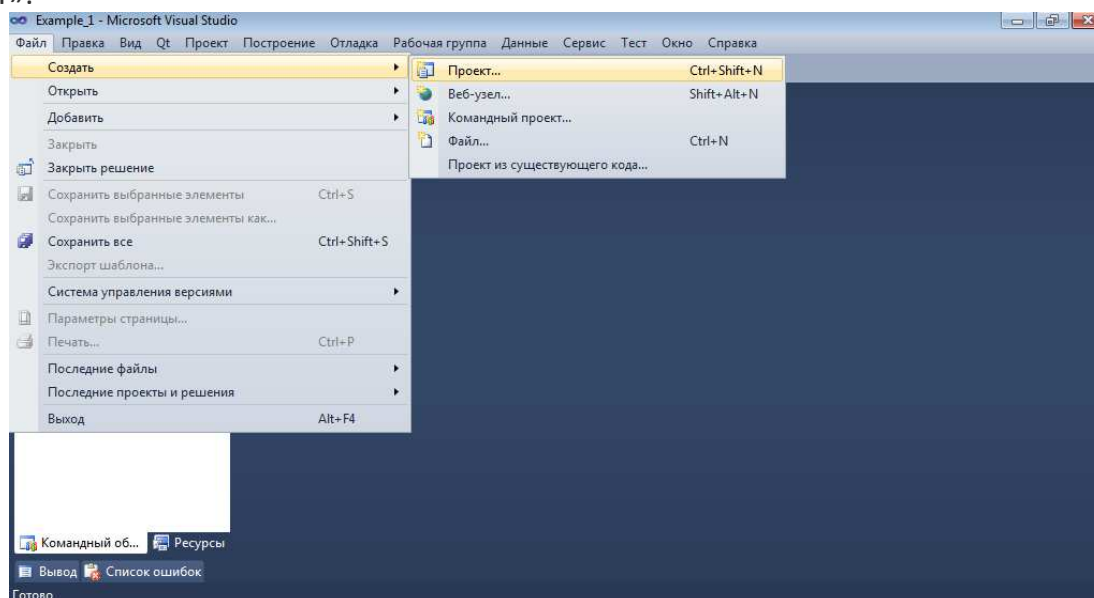
Как мы уже говорили, функции, образующие API, обычно, организованы в виде DLL - динамически подключаемых библиотек. Одно из достоинств DLL состоит в том, что, сколько бы приложений (процессов) не работало с функциями одной и той же DLL, код DLL существует в единственном экземпляре.

Win32 API (далее WinAPI) – это набор функций (*API – Application Programming Interface*), работающих под управлением ОС Windows. Они содержатся в библиотеке **windows.h**.

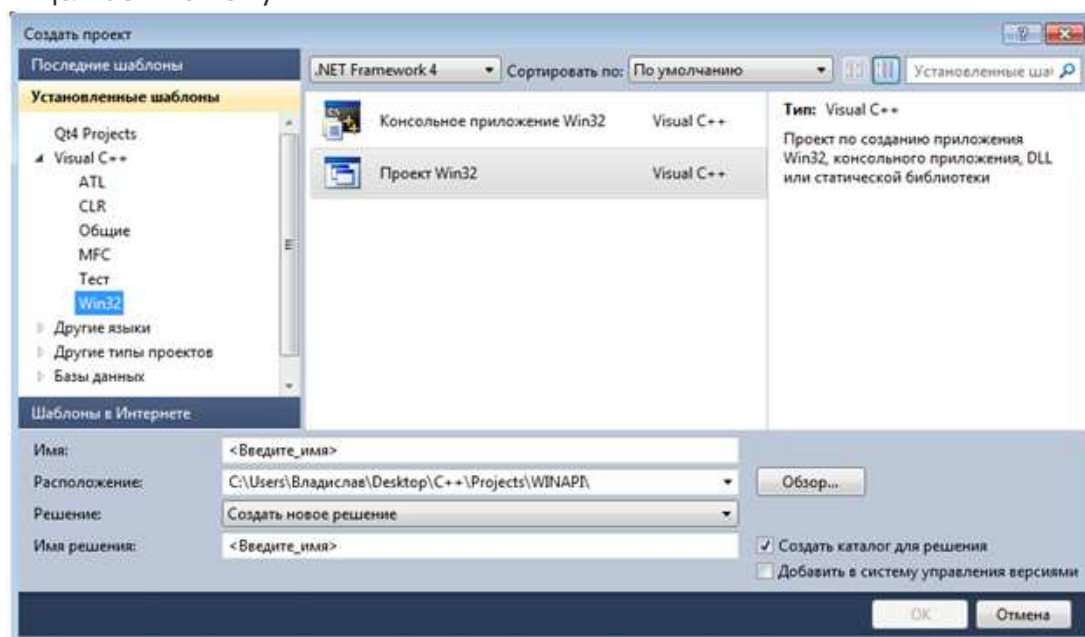
С помощью WinAPI можно создавать различные оконные процедуры, диалоговые окна, программы и даже игры. Эта, скажем так, библиотека является базовой в освоении программирования **Windows Forms**, MFC, потому что эти интерфейсы являются надстройками этой библиотеки. Освоив её, Вы без труда будете создавать формы, и понимать, как это происходит.

Не будем внедряться в теорию. Начнём с того, как создать этот проект в MVS, а в конце статьи будет разобран простой пример.

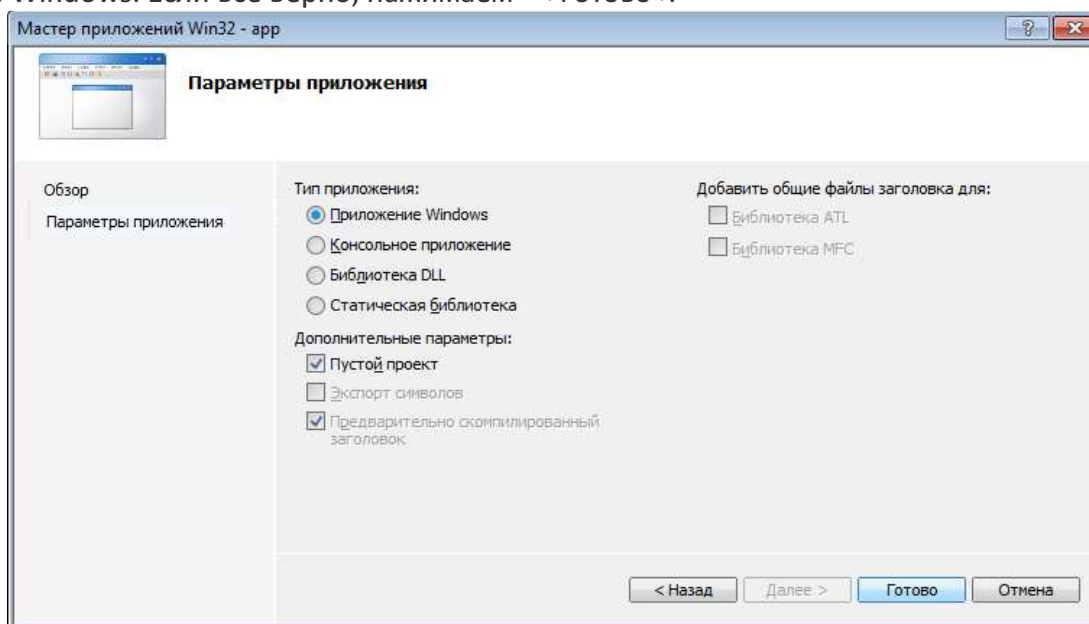
Итак. Сначала открываем Visual Studio, затем, нажимаем на вкладку «Файл», далее «Создать проект»:



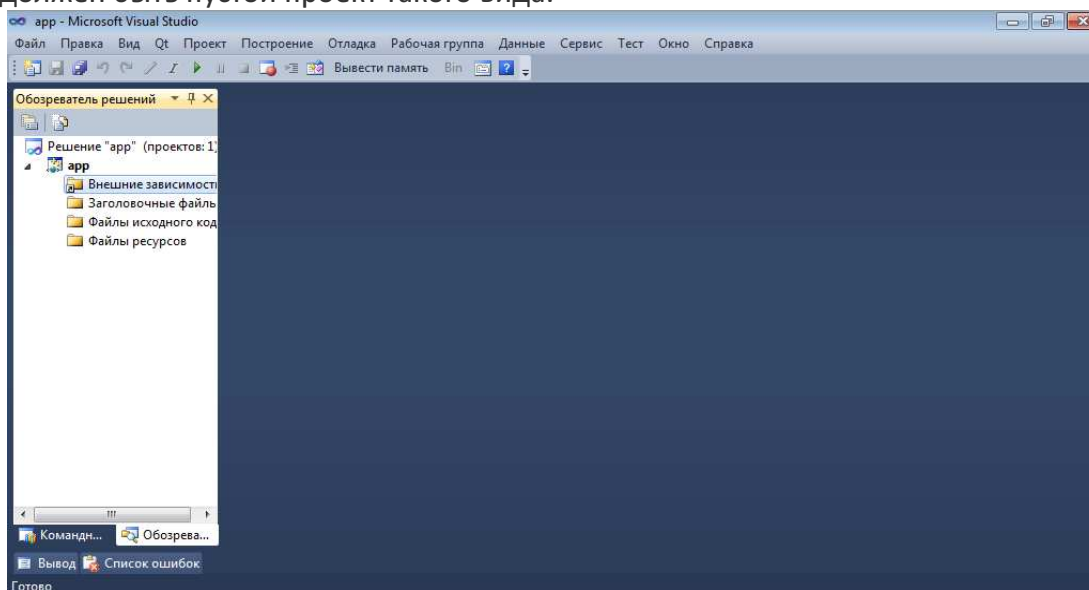
Затем, в раскрывающемся списке Visual C++ выбираем пункт Win32, там и будет «Проект Win32». Щелкаем по нему:



Вводим название проекта, указываем путь и нажимаем «ОК». Далее будет написано: «Добро пожаловать в мастер приложения Win32». Нажимаем далее. По-умолчанию у надписи «Пустой проект» галочка отсутствует. Нам нужно её поставить и убедиться, что у нас «Тип Приложения» — Приложение Windows. Если всё верно, нажимаем — «Готово».

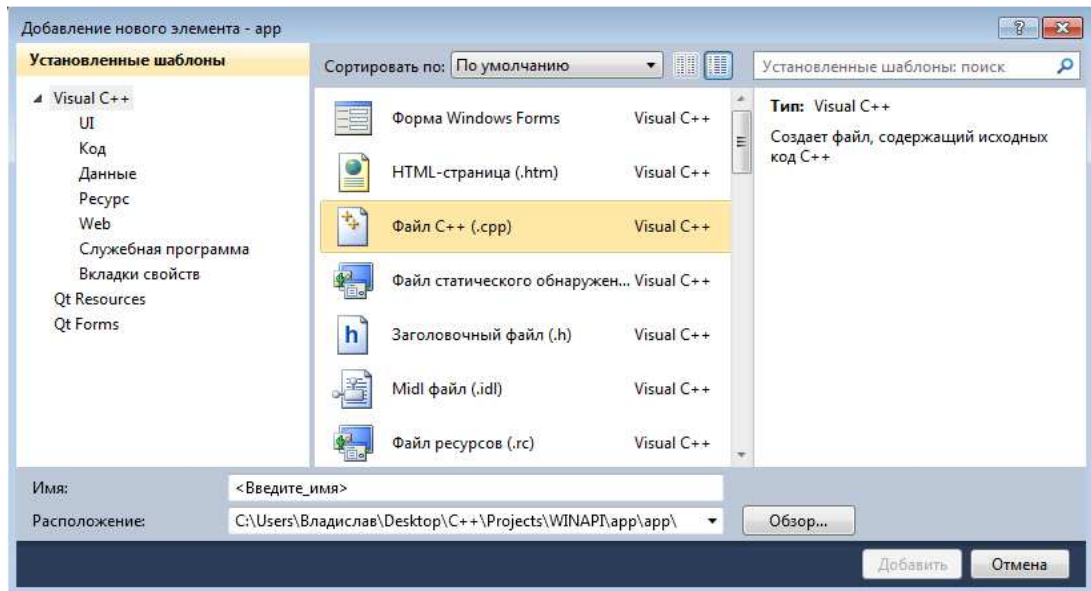


У нас должен быть пустой проект такого вида:



Ну а теперь начнём писать простую программу, которая традиционно будет выводить на экран надпись: «Привет, Мир!!!».

Естественно, к проекту нужно добавить файл типа «имя».cpp. Кликаем по «Файлы исходного кода» правой кнопкой мыши, в раскрывающемся списке выбираем вкладку — «Добавить», далее «Создать элемент...». В результате у нас должно появиться такое окно:



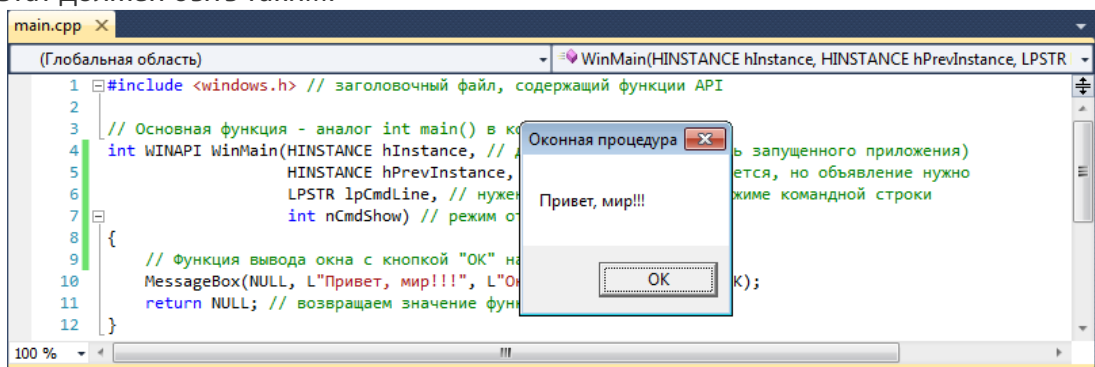
Выбираем «Файл С++», вводим имя, нажимаем «Добавить». Затем открываем этот файл и вставляем в него такой код (подробности далее):

```

1      #include <windows.h> // заголовочный файл, содержащий функции API
2
3      // Основная функция - аналог int main() в консольном приложении:
4      int WINAPI WinMain(HINSTANCE hInstance, // дескриптор экземпляра приложения
5                          HINSTANCE hPrevInstance, // в Win32 не используется
6                          LPSTR lpCmdLine, // нужен для запуска окна в режиме командной строки
7                          int nCmdShow) // режим отображения окна
8      {
9          // Функция вывода окна с кнопкой "ОК" на экран (о параметрах позже)
10         MessageBox(NULL, L"Привет, мир!!!", L"Оконная процедура", MB_OK);
11         return NULL; // возвращаем значение функции
12     }

```

Результат должен быть таким:



Теперь остановимся поподробнее на коде программы.

В первой строке мы подключаем заголовочный файл `windows.h`. В нём содержатся все необходимые «апишные» функции. Здесь всё понятно.

В 4-7 строках у нас описание функции `int WINAPI WinMain()`.

Квалификатор `WINAPI`, нужен для функции `WinMain` всегда. Просто запомните это. `WinMain` – название функции. Она имеет четыре параметра. Первый из них – `HINSTANCE hInstance` (строка 4). `hInstance` является дескриптором экземпляра окна (это некий код оконной процедуры, идентификатор, по которой ОС будет отличать её от остальных окон). Через него можно обращаться к окну в процессе работы в других функциях (об этом позже), что-либо менять в параметрах окна. `HINSTANCE` является одним из многочисленных типов данных определенных в `WinAPI`, таким же как `int`, например. А запись `HINSTANCE hInstance` говорит нам о том, что мы создаём новую переменную типа `HINSTANCE` с названием `hInstance`.

О типах данным мы поговорим позже, поэтому переходим к следующему параметру: **HINSTANCE hPrevInstance** (строка 5). Как написано в комментариях, в Win32 он не используется, так как он создан для 3.х разрядной системы, из предыдущего понятно, что это дескриптор экземпляра окна. Далее у нас переменная типа **LPSTR** (строка 6) с именем **lpCmdLine**. Она используется в том случае, если мы запускаем окно через командную строку с прописью параметров. Очень экзотический способ, поэтому мы не будем на нём задерживаться.

И последний параметр: целочисленный, определяет способ показа окна. Нужен для функции **ShowWindow**, которая будет описана позже. Например, с помощью него мы можем развернуть окно на весь экран, сделать его определённой высоты, прозрачным или поверх остальных.

Переходим к функции **MessageBox()** (строка 10). Она имеет четыре параметра и нужна для вывода сообщений о ошибках, например. В данном случае мы использовали её для вывода сообщения. В общем виде описание функции выглядит следующим образом:

```
1 int MessageBox(HWND hWnd, // дескриптор родительского окна
2 LPCTSTR lpText, // указатель на строку с сообщением
3 LPCTSTR lpCaption, // указатель на строку с текстом заголовка
4 UINT uType); // флаги для отображения кнопок, стиля пиктограммы и прочее
```

В нашем случае, первому параметру присвоен ноль. Всё потому, что у нас нет родительских окон (оно не запущено какой-нибудь программой).

Далее у нас идут две переменные типа **LPCTSTR**: **lpText** и **lpCaption**. Первая сообщает информацию, которая будет выведена в окне в текстовом виде. Вторая сообщает, что будет написано в тексте заголовка к окну. Это аналог **char *str**, но всё же нет. Для того, чтобы текст выводился корректно, нужно перед строкой поставить букву **L** (**UNICODE** строка).

Ну и последний тип данных – **UINT** – 32-х битное целое без знака. То есть аналог **unsigned int**. Этому параметру можно передавать некоторые значения (о них тоже позже), за счёт чего можно менять вид кнопки. В нашем случае – это **MB_OK** — означает, что окно создаёт кнопку с надписью «ОК» и соответствующим действием при её нажатии (закрытием приложения).

В строке 11 мы возвращаем значение функции, так как она имеет не тип **void**.

Типы данных в Win32 API

В WINAPI определено множество типов данных, так же, как и в C/C++ (**int**, **char**, **float** и т.д.). Учить их определения не обязательно. Достаточно помнить, что они существуют, а когда они появятся или потребуются где-нибудь в программе, посмотреть их определения. В дальнейшем мы будем использовать их все. Условно их можно разделить на несколько видов: основные, дескрипторные, строковые и вспомогательные.

Основные типы

С основными типами данных трудностей возникнуть не должно. Если всё же возникнут, то нужно заглянуть сюда <http://cppstudio.com/post/271/>.

BOOL – этот тип данных аналогичен **bool**. Он также имеет два значения – 0 или 1. Только при использовании WINAPI принято использовать вместо 0 спецификатор **NULL**. О нём ниже.

BYTE – байт, ну или восьмибитное беззнаковое целое число. Аналог – **unsigned char**.

DWORD – 32-битное беззнаковое целое. Аналоги: **unsigned long int**, **UINT**.

INT – 32-битное целое. Аналог – **long int**.

LONG – 32-битное целое – аналог всё также **long int**.

NULL – нулевой указатель. Вот его объявление:

```
1 void *NULL=0;
```

UINT – 32-битное беззнаковое целое. Аналоги: **unsigned long int**, **DWORD**.

Дескрипторные типы данных

Про дескрипторные типы немного рассказывалось на вводном уроке в WINAPI. Дескриптор, как говорилось ранее, — это идентификатор какого-либо объекта. Для разных типов объектов существуют разные дескрипторы. Дескриптор объекта можно описать так:

```
1 HANDLE h;
```

Есть также дескрипторы кисти, курсора мыши, шрифта и т.д. С их помощью мы можем при инициализации или в процессе работы приложения поменять какие-нибудь настройки, чего, например, мы не могли сделать в консольном приложении. Используются они в описательных функциях, управляющих типа: `CreateProcess()`, `ShowWindow()` и т.д. или как возвращаемое значение некоторых функций :

```
1 // получает дескриптор для устройства ввода или вывода:
2 HANDLE h = GetStdHandle(DWORD nStdHandle);
```

В этой функции мы получили дескриптор считывания потоков `std_in` и `std_out`. И можем, например, его использовать в каком-нибудь условии.

Не будем вдаваться в физику создания дескрипторов. Разве что, при необходимости или для большего понимания процессов.

Примечание: для удобства в WINAPI предусмотрены сокращения для типов данных. Первая буква **H** — означает, что это дескриптор, от слова `handle`.

HANDLE — дескриптор объекта.

HBITMAP — дескриптор растрового изображения. От фразы `handle bitmap`.

HBRUSH — дескриптор кисти. От фразы `handle brush`.

HCURSOR — дескриптор курсора. От фразы `handle cursor`.

HDC — дескриптор контекста устройства. От фразы `handle device context`.

HFONT — дескриптор шрифта. От фразы `handle font`.

HICONS — дескриптор криптограммы. От фразы `handle icons`.

HINSTANCE — дескриптор экземпляра приложения. От фразы `handle instance`.

HMENU — дескриптор меню. От фразы `handle menu`.

HPEN — дескриптор пера. От фразы `handle pen`.

HWND — дескриптор окна. От фразы `handle window`.

Строковые типы данных

Для начала начнём, с того, какие кодировки существуют в Windows ОС.

Есть два вида кодировок символов: *ANSI* и *UNICODE*. Однобайтные символы относятся к ANSI, двухбайтные — к кодировке UNICODE. Мы можем с лёгкостью подключить UNICODE кодировку в свойствах проекта. И тогда в коде создать переменную типа `char` можно будет так:

```
1 // создаём строку из 10 элементов:
2 wchar_t str[10];
```

Если же мы хотим использовать кодировку ANSI, то мы традиционно напишем:

```
1 // тоже создаём строку из 10 элементов:
2 char str[10];
```

В WINAPI, в зависимости от того, подключён юникод или нет, используются два вида строк UNICODE-ные или TCHAR-ные. Ниже описаны строковые типы данных.

Всё также для удобства, первые две буквы **LP** — от фразы `long pointer` сигнализируют о том, что это указатель.

LPCSTR — указатель на константную строку, заканчивающуюся нуль-терминатором. От фразы `long pointer constant string`.

LPCTSTR — указатель на константную строку, без UNICODE. От фразы `long pointer constant TCHAR string`. Это надстройка функции `LPCSTR`.

LPCWSTR — указатель на константную UNICODE строку. От фразы фразы `long pointer constant wide character string`. Это надстройка функции `LPCSTR`.

LPSTR – указатель на строку, заканчивающуюся нуль-терминатором. От фразы long pointer string.

LPTSTR – указатель на строку, без UNICODE. От фразы long pointer TCHAR string. Это надстройка функции LPSTR.

LPWSTR – указатель на UNICODE строку. От фразы long pointer wide character string. Это надстройка функции LPSTR.

TCHAR – символьный тип — аналог char и wchar_t.

Вспомогательные типы

Вспомогательные типы данных используются в некоторых функциях. В частности, параметры, описанные ниже, используются при работе с функцией обратного вызова оконной процедуры такого вида:

1 **LRESULT CALLBACK** ИмяФункции(**HWND** hWnd, **UINT** uMsg,

2 **WPARAM** wParam, **LPARAM** lParam);

Работа с данной функцией будет в следующих разделах.

LPARAM – тип для описания lParam (long parameter). Используются вместе с wParam в некоторых функциях.

LRESULT – значение, возвращаемое оконной процедурой имеет тип long.

WPARAM – тип для описания wParam (word parameter). Используются вместе с lParam в некоторых функциях.

На этом типы данных не закончены.

Создание полноценной оконной процедуры в Win32 API (Часть 1)

В разделе по WINAPI мы разобрали программу, которая выводит окно с сообщением. Кроме того, после нажатия «OK» окно закрывалось. И это всё, что она могла. Нетрудно догадаться, что при помощи Win32 API можно реализовывать более информативные окна. Поговорим о создании настоящей оконной процедуры.

Обобщённый алгоритм создания оконной процедуры в WINAPI:

1. Создать две функции: WinMain() – основная функция с параметрами, оговоренными в первой статье – аналог main в консоли, и функцию обрабатывающую процессы (название своё, пусть будет традиционно – WndProc()), потоки сообщений к/от ОС Windows.

2. Создать дескриптор окошка hMainWnd и зарегистрировать класс окошка WNDCLASSEX (то есть указать характеристики окна, которое будем создавать). Должен содержаться в WinMain.

3. Создать оболочку нашего окна. Должна содержаться в WinMain.

4. Создать циклы для обработки сообщений. Содержатся в функции WndProc.

5. Создать функцию для вывода окна ShowWindow и другие вспомогательные.

Теперь будет трудно, потому что сейчас появятся различные функции с кучей параметров, классы, которые следует знать, понимать, ну и некоторые другие премудрости языков C и C++.

Для начала опишем общую структуру функции **WinMain()**:

```
1        int WINAPI WinMain(HINSTANCE hInst,
2                        HINSTANCE hPreviousInst,
3                        LPSTR lpCommandLine,
4                        int nCommandShow
5                        )
6        {
7            // создаём дескриптор окна
8            // описываем класс окна
9            // создаём окошко, показываем его на экране
10            // возвращаем значение при неудаче или при выходе
```

```
11     }
```

Для начала создадим дескриптор окна:

```
1     HWND hMainWnd; // создаём дескриптор будущего окошка
2     // и т.д.
```

Чтобы инициализировать поля класса окна, нужно создать его экземпляр, далее через него заполнить поля класса.

Объявление этого класса в `windows.h` выглядит так:

```
1     struct tagWNDCLASSEX{
2         UINT cbSize; // величина структуры (в байтах)
3         UINT style; // стиль класса окошка
4         WNDPROC WndProc; // указатель на имя пользовательской функции
5         int cbWndExtra; // число освобождаемых байтов в конце структуры
6         int cbClsExtra; // число освобождаемых байтов при создании экземпляра приложения
7         HICON hIcon; // дескриптор значка
8         HICON hIconMini; // .... маленького значка (в трее)
9         HCURSOR hCursor; // .... курсора мыши
10        HBRUSH hbrBack; // .... цвета фона окошка
11        HINSTANCE hInst; // .... экземпляра приложения
12        LPCTSTR lpszClassName; // указатель на const-строку, содержащую имя класса
13        LPCTSTR lpszMenuName; // указатель на const-строку, содержащую имя меню, применяемого для класса
14    }WNDCLASSEX;
```

То есть мы должны создать переменную типа `WNDCLASSEX`, обычно это `wc`, затем через неё инициализировать поля класса, примерно так:

```
1     // создавали дескриптор окна
2     WNDCLASSEX wc; // создаём экземпляр, для обращения к членам класса WNDCLASSEX
3     wc.cbSize = sizeof(wc); // размер структуры (в байтах)
4     wc.lpfnWndProc = WndProc; // указатель на пользовательскую функцию
5     // и т.д.
```

Это необходимо, чтобы использовать этот класс в дальнейшем (если мы захотим). Это будет шаблоном для создания кучи окон. Конечно, на первых порах нам не нужно столько окошек. Но регистрация нужна однозначно! Это - формальная сторона. Мы даже можем задать параметры по умолчанию при инициализации полей, а не придумывать каким должно быть окно (о параметрах, которыми должны инициализировать поля класса мы обсудим в следующем уроке).

Итак, ещё раз: **если хоть одно поле класса не будет инициализировано, то окно не создается**. Для проверки существует полезная функция `RegisterClassEx()`. Это и есть следующий этап после заполнения полей класса `WNDCLASSEX`: обязательная проверка регистрации класса:

```
1     // регистрировали класс
2     if(!RegisterClassEx(&wc)){
3         // в случае отсутствия регистрации класса:
4         MessageBox(NULL,
5             L"Не получилось зарегистрировать класс!",
6             L"Ошибка", MB_OK);
7         return NULL; // возвращаем, следовательно, выходим из WinMain
8     }
9     // и т.д.
```

Как видим, в этом коде всё просто. Если `RegisterClassEx()` возвращает адекватный `ATOM` (таково описание:

```
1     ATOM WINAPI RegisterClassEx(const WNDCLASSEX *lpWindowClass);
```

), то мы выводим соответствующее сообщение, которое мы разбирали в первой статье. Её параметр – это ссылка на экземпляр класса `WNDCLASSEX wc`.

Следующее, что мы обязаны сделать в функции **WinMain()** – вызвать функцию **CreateWindow()** и присвоить её значение дескриптора, который мы создавали на первом этапе. Примерно это так (описание всех этих параметров будет в следующем уроке):

```
1 // проверяли, зарегистрирован ли класс
2 hMainWnd = CreateWindow(szClassName, // имя класса
3     L"Полноценная оконная процедура", // имя окна (то что сверху)
4     WS_OVERLAPPEDWINDOW | WS_VSCROLL, // режимы отображения окошка
5     CW_USEDEFAULT, // положение окна по оси x (по умолчанию)
6     NULL, // позиция окна по оси y (раз дефолт в x, то писать не нужно)
7     CW_USEDEFAULT, // ширина окошка (по умолчанию)
8     NULL, // высота окна (раз дефолт в ширине, то писать не нужно)
9     HWND(NULL), // дескриптор родительского окошка (у нас нет род. окон)
10    NULL, // дескриптор меню (у нас его нет)
11    HINSTANCE(hInst), // .... экземпляра приложения
12    NULL); // ничего не передаём из WndProc
13 // и т.д.
```

Описание **CreateWindow()** в **windows.h** выглядит так:

```
1 HWND CreateWindow(
2     LPCTSTR lpClassName, // имя нашего класса
3     LPCTSTR lpWindowName, // название окошка (надпись сверху)
4     DWORD dwStyle, // стиль окошка
5     int x, // позиция окошка по оси x
6     int y, // позиция окна по оси y (отсчёт вниз)
7     int nWidth, // ширина окошка
8     int nHeight, // высота окошка
9     HWND hWndParent, // идентификатор родительского окошка
10    HMENU hMenu, // .....меню
11    HINSTANCE hInst, // дескриптор экз-ра прил-ния
12    LPVOID lParam
13    ); // указатель на данные, передаваемые из пользовательской функции
```

В случае удачи, функция в переменную **hMainWnd** возвратит не **NULL**. Наверное, не трудно догадаться, что надо сделать проверку (по аналогии с **RegisterClassEx()**):

```
1 // создали окно
2 if(!hMainWnd){
3     // в случае некорректного создания окна (неверные параметры и тп):
4     MessageBox(NULL, L"Не получилось создать окно!", L"Ошибка", MB_OK);
5     return NULL; // выходим из приложения
6 }
7 // и т.д.
```

После чего нам необходимо вызвать две функции:

```
1 // проверяли, создано ли окно
2 ShowWindow(hMainWnd, nCommandShow);
3 UpdateWindow(hMainWnd);
4 // и т.д.
```

Подробно описывать их нет надобности, так как имеют несколько параметров.

Первая функция отображает окно на экране ПК. Её первый параметр – дескриптор окошка (он возвращался **CreateWindow()**). Второй параметр – стиль отображения. При первом запуске окна должен быть равен последнему параметру функции **WinMain()**, а в последующие разы можно вписывать свои данные (об этом в следующих статьях).

Вторая функция – видно по названию, отвечает за обновления окошка на экране при сворачиваниях или при динамической информации. Его параметр – всё тот же дескриптор окна.

В функции остался цикл и возвращение значение функции:

```
1 // показывали, обновляли окно
2 while(GetMessage(&msg, NULL, NULL, NULL)){
3     TranslateMessage(&msg);
4     DispatchMessage(&msg);
5 }
6 return msg.wParam;
7 // всё!
```

Так как объекты структуры **MSG** связаны с рассылкой сообщений системному окну и наоборот и это всё содержится в пользовательской функции **WndProc**, поэтому описание этих функций будет в следующем уроке.

Напоследок модифицированная программа, выводящая окно с сообщением (из первой статьи) с другими кнопками:

```
1 #include <windows.h> // содержит API
2
3 // Основная функция:
4 int WINAPI WinMain (HINSTANCE hInst, // дескриптор экземпляра приложения
5                     HINSTANCE hPreviousInst, // в Win32 не используется, но объявление нужно
6                     LPSTR lpCommandLine, // нужен для запуска окошка в режиме командной строки
7                     int nCommandShow) // режим отображения окна
8 {
9     int result = MessageBox(NULL, L"Вам нравится WINAPI?!", L"Задача",
10                            MB_ICONQUESTION | MB_YESNO);
11     switch (result)
12     {
13     case IDYES: MessageBox (NULL, L"Продолжайте в том же духе!!!",
14                             L"Ответ", MB_OK | MB_ICONASTERISK); break;
15     case IDNO:  MessageBox (NULL, L"Очень жаль!!!", L"Ответ",
16                             MB_OK | MB_ICONSTOP); break;
17     }
18     return NULL;
19 }
```

Скомпилируйте данный код, и Вы обнаружите, что с функцией **MessageBox()** тоже можно поэкспериментировать.

Черточки между параметрами в **строках 10,14 и 17** означают, что параметры используются вместе.

Третьим параметром мы можем записать следующие идентификаторы:

параметры кнопок:

MB_ABORTRETRYIGNORE — три кнопки: ABORT, RETRY, IGNORE
MB_CANCELTRYCONTINUE — три кнопки: CANCEL, TRY, CONTINUE
MB_HELP MB_OK MB_OKCANCEL — 2 кнопки: OK, CANCEL
MB_RETRYCANCEL — 2 кнопки: RETRY, CANCEL
MB_YESNO — 2 кнопки: YES, NO
MB_YESNOCANCEL — три кнопки: YES, NO, CANCEL

параметры пиктограммы:

MB_ICONSTOP — выводит крестик **MB_ICONQUESTION** —знак вопроса
MB_ICONEXCLAMATION —восклицательный знак в треугольнике
MB_ICONASTERISK —восклицательный знак

А возвращать значения при нажатии вышеуказанных кнопок функция **MessageBox** будет такие:

IDABORT — при нажатии на ABORT

IDCANCEL –на кнопку CANCEL
IDCONTINUE –на кнопку CONTINUE
IDIGNORE –на кнопку IGNORE
IDNO –на кнопку NO
IDOK –на кнопку OK
IDRETRY –на кнопку RETRY
IDTRYAGAIN –на кнопку TRY AGAIN
IDYES –на кнопку YES

Создание полноценной оконной процедуры в Win32 API (Часть 2)

Ранее мы на половину закончили разработку полноценной оконной процедуры. В этой статье она будет закончена. Итак, мы остановились на функции `WinMain()`. В её последних строках содержались вызовы двух функций, взаимодействующих с нашей функцией `WndProc()` (о ней здесь и будет идти речь) и оператор возврата значения функции. Конструкция функции `WndProc()` должна быть условно такая:

```
1     LRESULT CALLBACK WndProc(HWND hWnd, // дескриптор окошка
2         UINT uMsg, // сообщение, посылаемое ОС
3         WPARAM wParam, // параметры
4         LPARAM lParam) // сообщений, для последующего обращения
5     {
6         // 1) создаём нужные переменные
7         // 2) расписываем условия, при которых нужно выполнить нужное действие (switch)
8         // 3) Возвращаем значение функции
9     }
```

`LRESULT` – это возвращаемое значение. `CALLBACK` нужно писать, так как это функция обратного вызова. Сначала создаём необходимые переменные. Для начала создадим экземпляр контекста устройства `HDC` для нужной ориентации текста в окне. Кроме того, для обработки области окна нам нужны ещё две переменные `RECT` (прямоугольная область окна) и `PAINTSTRUCT` (в структуре информация об окне для прорисовки) соответственно:

```
1     // создаём нужные переменные:
2     HDC hDC;
3     PAINTSTRUCT ps;
4     RECT rect;
5     // и т.д.
```

Чтобы поменять цвет текста (и не только) нужно создать переменную типа `COLORREF` и присвоить ей возвращаемое значение функции `RGB()` (от англ. Red Green Blue) с тремя параметрами:

```
1     // создавали нужные переменные
2     COLORREF colorText = RGB(255, 0, 0); // параметры int
3     // и т.д.
```

Эта функция преобразовывает целочисленный тип в интенсивность цвета и возвращает значение смешивания трёх цветов (интенс. красн. + интенс. зел.+ интенс. син.). Как видно с помощью градаций этих цветов, можно создать $255 \times 255 \times 255 = 16'581'375$ цветов. ОС Windows и различные функции наших программ ежесекундно отправляют тысячи сообщений каждому отдельному приложению. Эти сообщения могут быть отправлены из-за нажатия клавиши или нажатия кнопки на мыши и т.д. Для этих случаев у нас существует структура `MSG`, описанная в `WinMain()`, которая хранит информацию об этих сообщениях. В `WndProc()` есть условия выбора для этих сообщений. Если ОС отправляет сообщение `WM_PAINT`, например, то в окне должно что-то рисоваться. Эти сообщения обрабатываются условием `switch()` (оператор множественного выбора), параметрами которого являются `uMsg`. `uMsg` мы создали при описании нашей

функции `WndProc()`. Основное значение в этом операторе, без которого окно не будет обновляться после сворачивания и не закроется: `WM_DESTROY`. Также есть `WM_PAINT`, который нужен для прорисовки в клиентской области. WM – от слов Window Message. То есть:

```
1      // создавали переменные
2      switch(uMsg){
3          case WM_PAINT:
4              // что то рисуем
5          case WM_DESTROY:
6              // обязательно делаем условие закрытия окошка
7          default:
8              return DefWindowProc(hWnd, uMsg, wParam, lParam); // об этом далее
9      }
```

И что же мы должны делать в этих «кейсах»? При посылке сообщения `WM_PAINT` – вызываем функции рисования `BeginPaint()`, `GetClientRect()`, `SetTextColor()`, `DrawText()`, `EndPaint()`. По названию видно, что эти функции делают. Функция `BeginPaint()` в прямом смысле начинает рисовать. Только для этого ей нужно иметь дескриптор окна и объект `PAINTSTRUCT` (у нас это `ps`). Она возвращает значение типа `HDC`, поэтому нам нужно присвоить ей `hDc`. `GetClientRect()` выбирает область. Параметры у неё аналогичны предыдущей функции: дескриптор окна и указатель на объект класса `RECT` (у нас это `rect`). Функция `SetTextColor()` возвращает цвет текста. Её параметры: возвращаемое значение функции `BeginPaint()` – `hDC` и указатель на объект класса `COLORREF`. Мы могли и не задавать отдельно цвет текста, создавая при этом переменную `colorText`, а могли сделать это прямо в ней. Но с точки зрения читаемости кода и его понятливости – это в корне не правильно. Старайтесь всегда объявлять переменные отдельно и писать в комментариях, зачем они нужны и тогда не будет вопросов, какие параметры имеет функция, спустя год как вы последний раз закрыли проект по WinAPI. Также соблюдайте венгерскую нотацию по программированию, суть которой: имена переменных должны нести смысл их существования и показывать тип данных. Объявление функции `DrawText()`:

```
1      int DrawText(
2          HDC hDc, // дескриптор контекста устр-ва
3          LPCTSTR lpchText, // указатель на нашу строку
4          int nCount, // длина текста (если равно -1, то определяет сам)
5          LPRECT lpRect, // указатель на объект RECT
6          UINT uFormat // формат отображения текста
7      );
```

На счёт первых 4х параметров всё ясно. Четвёртый `uFormat` – имеет несколько видов. Обычно используются `DT_SINGLELINE`, `DT_CENTER` и `DT_VCENTER` для отображения текста в центре области, в одну линию. Но Вы можете задать другие параметры (о чём поговорим в следующих уроках). Функция `EndPaint()` имеет два параметра: дескриптор окна и объект `ps`. Заметили аналогию с `BeginPaint()`? Что делать при вызове `WM_PAINT`, мы знаем (не забываем в конце дописать `break`). `WM_DESTROY` посылается окошку функцией `DestroyWindow()`, которая вызывается в случае, если мы его закрыли. А это происходит в операторе `default`. При этом происходит вызов функции `DefWindowProc()`, параметры которой те же, что и у `WndProc()`. В своём теле `WM_DESTROY` должен иметь функцию `PostQuitMessage()`, которая посылает `WinMain()` сообщение `WM_QUIT`. Её параметр обычно `NULL`, интерпретирующийся для главной функции `WinMain()`, как `WM_QUIT`.

```
VOID WINAPI PostQuitMessage(int nExitCode);
```

По завершению `switch()` мы возвращаем нулевое значение функции `WndProc()`. А теперь вернёмся к `WinMain()`, в частности к обработчику сообщений:

```
1      while(GetMessage(&msg, NULL, NULL, NULL)){
2          TranslateMessage(&msg);
3          DispatchMessage(&msg);
```

```

4     }
5     return msg.wParam;

```

Функция **GetMessage()** имеет такое описание:

```

1     BOOL WINAPI GetMessage(LPMSG lpMsg, // указатель на структуру MSG
2                               HWND hWnd, // дескриптор окошка
3                               UINT wMsgFilterMin, // фильтры
4                               UINT wMsgFilterMax // фильтры для выборки сообщений
5                               );

```

Она обрабатывает сообщения, посылаемые ОС. Первый параметр – это адрес структуры **MSG**, в которую помещается очередное сообщение. Второй параметр - дескриптор окна. Третий и четвёртый параметры указывают порядок отбора сообщений. Обычно это нулевые значения и функция отбирает любые значения из очереди. Цикл прекращается, если она получает сообщение **WM_QUIT**. В таком случае она возвращает **FALSE** и мы выходим из программы. Функции **TranslateMessage()** и **DispatchMessage()** в цикле нужны для интерпретации самих сообщений. Обычно это используется при обработке нажатых кнопок на клавиатуре. В следующих уроках мы познакомимся с этими хитростями. По окончании цикла мы возвращаем ОС код возврата **msg.wParam**. В итоге у нас должен получиться такой код:

```

1     #include <windows.h> // заголовочный файл, содержащий WINAPI
2
3     // Прототип функции обработки сообщений с пользовательским названием:
4     LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
5     TCHAR mainMessage[] = L"Какой то-текст!"; // строка с сообщением
6
7     // Управляющая функция:
8     int WINAPI WinMain(HINSTANCE hInst, // дескриптор экземпляра приложения
9                       HINSTANCE hPrevInst, // не используем
10                      LPSTR lpCmdLine, // не используем
11                      int nCmdShow) // режим отображения окошка
12     {
13         TCHAR szClassName[] = L"Мой класс"; // строка с именем класса
14         HWND hMainWnd; // создаём дескриптор будущего окошка
15         MSG msg; // создаём экземпляр структуры MSG для обработки сообщений
16         WNDCLASSEX wc; // создаём экземпляр, для обращения к членам класса WNDCLASSEX
17         wc.cbSize      = sizeof(wc); // размер структуры (в байтах)
18         wc.style       = CS_HREDRAW | CS_VREDRAW; // стиль класса окошка
19         wc.lpfnWndProc  = WndProc; // указатель на пользовательскую функцию
20         wc.lpszMenuName = NULL; // указатель на имя меню (у нас его нет)
21         wc.lpszClassName = szClassName; // указатель на имя класса
22         wc.cbWndExtra   = NULL; // число освобождаемых байтов в конце структуры
23         wc.cbClsExtra   = NULL; // число освобождаемых байтов при создании экземпляра приложения
24         wc.hIcon        = LoadIcon(NULL, IDI_WINLOGO); // дескриптор пиктограммы
25         wc.hIconSm      = LoadIcon(NULL, IDI_WINLOGO); // дескриптор маленькой пиктограммы (в трее)
26         wc.hCursor      = LoadCursor(NULL, IDC_ARROW); // дескриптор курсора
27         wc.hbrBackground = (HBRUSH)GetStockObject(WHITE_BRUSH); // дескриптор кисти для закраски фона окна
28         wc.hInstance    = hInst; // указатель на строку, содержащую имя меню, применяемого для класса
29         if(!RegisterClassEx(&wc)){
30             // в случае отсутствия регистрации класса:
31             MessageBox(NULL, L"Не получилось зарегистрировать класс!", L"Ошибка", MB_OK);
32             return NULL; // возвращаем, следовательно, выходим из WinMain
33         }
34         // Функция, создающая окошко:
35         hMainWnd = CreateWindow(
36             szClassName, // имя класса
37             L"Полноценная оконная процедура", // имя окошка (то что сверху)

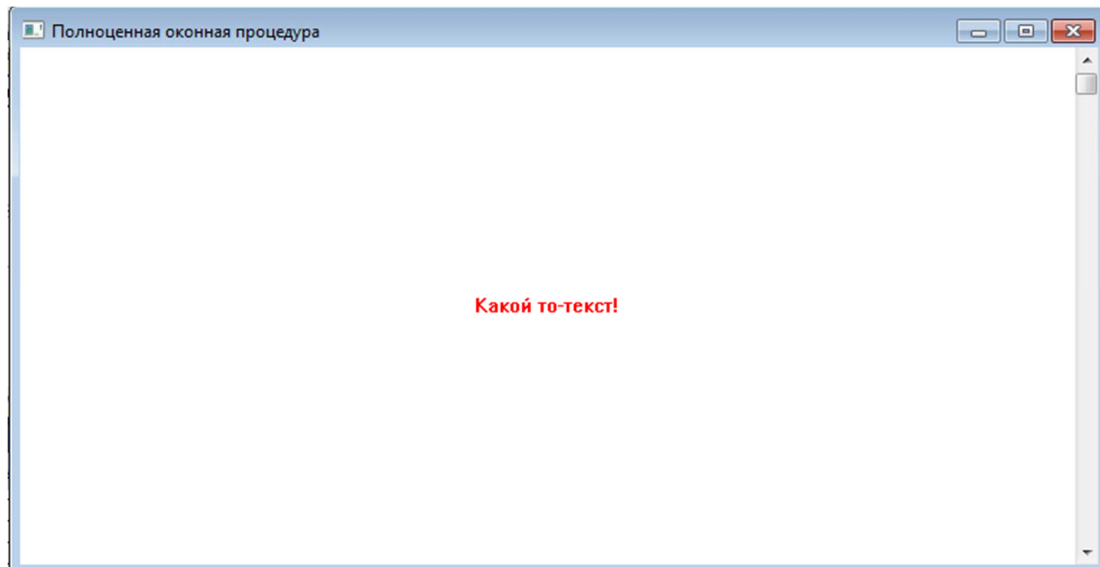
```

```

38     WS_OVERLAPPEDWINDOW | WS_VSCROLL, // режимы отображения окошка
39     CW_USEDEFAULT, // позиция окошка по оси x
40     NULL, // позиция окошка по оси y (раз дефолт в x, то писать не нужно)
41     CW_USEDEFAULT, // ширина окошка
42     NULL, // высота окошка (раз дефолт в ширине, то писать не нужно)
43     (HWND)NULL, // дескриптор родительского окна
44     NULL, // дескриптор меню
45     HINSTANCE(hInst), // дескриптор экземпляра приложения
46     NULL); // ничего не передаём из WndProc
47 if(!hMainWnd){
48     // в случае некорректного создания окошка (неверные параметры и тп):
49     MessageBox(NULL, L"Не получилось создать окно!", L"Ошибка", MB_OK);
50     return NULL;
51 }
52 ShowWindow(hMainWnd, nCmdShow); // отображаем окошко
53 UpdateWindow(hMainWnd); // обновляем окошко
54 while(GetMessage(&msg, NULL, NULL, NULL)){ // извлекаем сообщения из очереди, посылаемые функцией
55     TranslateMessage(&msg); // интерпретируем сообщения
56     DispatchMessage(&msg); // передаём сообщения обратно ОС
57 }
58 return msg.wParam; // возвращаем код выхода из приложения
59 }
60
61 LRESULT CALLBACK WndProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam){
62     HDC hDC; // создаём дескриптор ориентации текста на экране
63     PAINTSTRUCT ps; // структура, сод-щая информацию о клиентской области (размеры, цвет и тп)
64     RECT rect; // стр-па, определяющая размер клиентской области
65     COLORREF colorText = RGB(255, 0, 0); // задаём цвет текста
66     switch(uMsg){
67     case WM_PAINT: // если нужно нарисовать, то:
68         hDC = BeginPaint(hWnd, &ps); // инициализируем контекст устройства
69         GetClientRect(hWnd, &rect); // получаем ширину и высоту области для рисования
70         SetTextColor(hDC, colorText); // устанавливаем цвет контекстного устройства
71         DrawText(hDC, mainMessage, -1, &rect, DT_SINGLELINE|DT_CENTER|DT_VCENTER); // рисуем текст
72         EndPaint(hWnd, &ps); // заканчиваем рисовать
73         break;
74     case WM_DESTROY: // если окошко закрылось, то:
75         PostQuitMessage(NULL); // отправляем WinMain() сообщение WM_QUIT
76         break;
77     default:
78         return DefWindowProc(hWnd, uMsg, wParam, lParam); // если закрыли окошко
79     }
80     return NULL; // возвращаем значение
81 }

```

А после компиляции такое окошко:



Ввод и вывод данных Win32 API

В этой статье мы научимся обрабатывать данные, поступающие с клавиатуры. В дальнейшем это будет полезным пособием при создании простейшего текстового редактора типа notepad.exe.

Ниже представлен листинг:

```
1  #include <windows.h>
2
3  LRESULT CALLBACK WindowProcess(HWND, UINT, WPARAM, LPARAM);
4
5  int WINAPI WinMain(HINSTANCE hInst,
6                    HINSTANCE hPrevInst,
7                    LPSTR pCommandLine,
8                    int nCommandShow){
9      TCHAR className[] = L"Мой класс";
10     HWND hWindow;
11     MSG message;
12     WNDCLASSEX windowClass;
13
14     windowClass.cbSize    = sizeof(windowClass);
15     windowClass.style     = CS_HREDRAW | CS_VREDRAW;
16     windowClass.lpfnWndProc = WindowProcess;
17     windowClass.lpszMenuName = NULL;
18     windowClass.lpszClassName = className;
19     windowClass.cbWndExtra  = NULL;
20     windowClass.cbClsExtra  = NULL;
21     windowClass.hIcon       = LoadIcon(NULL, IDI_WINLOGO);
22     windowClass.hIconSm     = LoadIcon(NULL, IDI_WINLOGO);
23     windowClass.hCursor     = LoadCursor(NULL, IDC_ARROW);
24     windowClass.hbrBackground = (HBRUSH)GetStockObject(WHITE_BRUSH);
25     windowClass.hInstance   = hInst;
26
27     if(!RegisterClassEx(&windowClass))
28     {
29         MessageBox(NULL, L"Не получилось зарегистрировать класс!", L"Ошибка", MB_OK);
30         return NULL;
31     }
32     hWindow = CreateWindow(className,
```

```

33         L"Программа ввода символов",
34         WS_OVERLAPPEDWINDOW,
35         CW_USEDEFAULT,
36         NULL,
37         CW_USEDEFAULT,
38         NULL,
39         (HWND)NULL,
40         NULL,
41         HINSTANCE(hInst),
42         NULL
43     );
44     if(!hWindow){
45         MessageBox(NULL, L"Не получилось создать окно!", L"Ошибка", MB_OK);
46         return NULL;
47     }
48     ShowWindow(hWindow, nCommandShow);
49     UpdateWindow(hWindow);
50     while(GetMessage(&message, NULL, NULL, NULL)){
51         TranslateMessage(&message);
52         DispatchMessage(&message);
53     }
54     return message.wParam;
55 }
56
57 LRESULT CALLBACK WindowProcess(HWND hWindow,
58     UINT uMessage,
59     WPARAM wParam,
60     LPARAM lParam)
61 {
62     HDC hDeviceContext;
63     PAINTSTRUCT paintStruct;
64     RECT rectPlace;
65     HFONT hFont;
66     static char text[2]={' ', '\0'};
67     switch (uMessage)
68     {
69     case WM_CREATE:
70         MessageBox(NULL,
71             L"Пожалуйста, вводите символы и они будут отображаться на экране!",
72             L"ВНИМАНИЕ!!!", MB_ICONASTERISK|MB_OK);
73         break;
74     case WM_PAINT:
75         hDeviceContext = BeginPaint(hWindow, &paintStruct);
76         GetClientRect(hWindow, &rectPlace);
77         SetTextColor(hDeviceContext, NULL);
78         hFont=CreateFont(90,0,0,0,0,0,0,0,
79             DEFAULT_CHARSET,
80             0,0,0,0,
81             L"Arial Bold"
82             );
83         SelectObject(hDeviceContext,hFont);

```

```

84     DrawText(hDeviceContext, (LPCWSTR)text, 1, &rectPlace, DT_SINGLELINE|DT_CENTER|DT_VCENTER);
85     EndPaint(hWindow, &paintStruct);
86     break;
87 case WM_KEYDOWN:
88     switch (wParameter)
89     {
90     case VK_HOME:case VK_END:case VK_PRIOR:
91     case VK_NEXT:case VK_LEFT:case VK_RIGHT:
92     case VK_UP:case VK_DOWN:case VK_DELETE:
93     case VK_SHIFT:case VK_SPACE:case VK_CONTROL:
94     case VK_CAPITAL:case VK_MENU:case VK_TAB:
95     case VK_BACK:case VK_RETURN:
96         break;
97     default:
98         text[0]=(char)wParameter;
99         InvalidateRect(hWindow, NULL, TRUE);
100        break;
101    }break;
102 case WM_DESTROY:
103     PostQuitMessage(0);
104     break;
105 default:
106     return DefWindowProc(hWindow, uMessage, wParameter, lParameter);
107 }
108 return NULL;
109 }

```

Данная программа выводит цифры и буквы, при этом не реагирует на нажатие дополнительных клавиш, типа enter, shift и т.д.

В функции `WinMain()` ничего нового. А в `WindowProcess()` мы добавили обработчики событий нажатия клавиши.

Нажатие любой клавиши автоматически посылает системе аппаратное сообщение, которое обрабатывается и высылается приложению. За эту операцию отвечает сообщение `WM_KEYDOWN`.

Его описание сложно и на данном этапе не имеет смысла, так как мы не включили кириллицу. При посылке этого сообщения в `wParameter` записывается значение нажатой клавиши. Конечно же, всё зависит от клавиатуры, раскладки. Поэтому всего кодов для клавиш много и мы не будем описывать, что делать при нажатии той или иной, а просто исключим значения не нужных нам клавиш.

С 87-й по 101-ю строки мы обрабатываем сообщение `WM_KEYDOWN` (нажатие клавиши). Сначала мы создаём переключатель `switch` для значения `wParameter`, куда записан идентификатор клавиши. С 90-й по 95-ю строки мы исключаем значения home, end, page up, page down, стрелка влево, стрелка вправо, стрелка вверх, стрелка вниз, delete, shift, space, ctrl, caps lock, alt, tab, backspace, enter соответственно. Возможно, на вашей клавиатуре есть ещё клавиши, вы можете также их исключить при необходимости, потому что при нажатии будет «абракадабра». То есть если в идентификаторе `wParameter` у нас эти значения, мы ничего не делаем – в 96-й строке оператор `break`.

А под значениями `default` у нас выполняется преобразование. В 66 строке мы создали статическую строку типа `char` из 2-х элементов. Статическая она, потому что в процессе работы программы мы неоднократно выходим из `WindowProcess()` в цикл обработки с `GetMessage()`, расположенный в `WinMain()`, а значение клавиши нам нужно сохранять. Ну а в 98 строке мы инициализируем её первый элемент `wParameter`-ом.

В следующей строке мы вызываем функцию `InvalidateRect()`, после чего выходим из `WM_KEYDOWN`. Первым параметром идёт – дескриптор окна, вторым – область будущего «затирания» окна (`NULL` – всё окно перерисовывается в `WM_PAINT`), третьим параметром идёт «затирание» фона (если `TRUE`, то стираем). При этом мы автоматически посылаем приложению `message WM_PAINT` и окно перерисовывается заново. Для чего это нужно?

Как мы помним в случаях сворачивания, перемещения окна вызывается `WM_PAINT`. Нам же это нужно сделать явно, то есть сделать окно недействительным, для того, чтобы сразу вывести значение нажатой клавиши на экран. Иначе она появится только тогда, когда мы начнём манипулировать окном.

Теперь возвратимся 65-й строке. В ней объявлен экземпляр класса `HFONT`, который отвечает за шрифт, его размер и другие премудрости. Он будет использоваться в `WM_PAINT`.

В 69-й строке есть обработчик сообщения `WM_CREATE`. Он срабатывает, когда создаётся окно, то есть вызывается функция `CreateWindow()`. На этом этапе решено создать дополнительное окно с сообщением для удобства (строки 70-72).

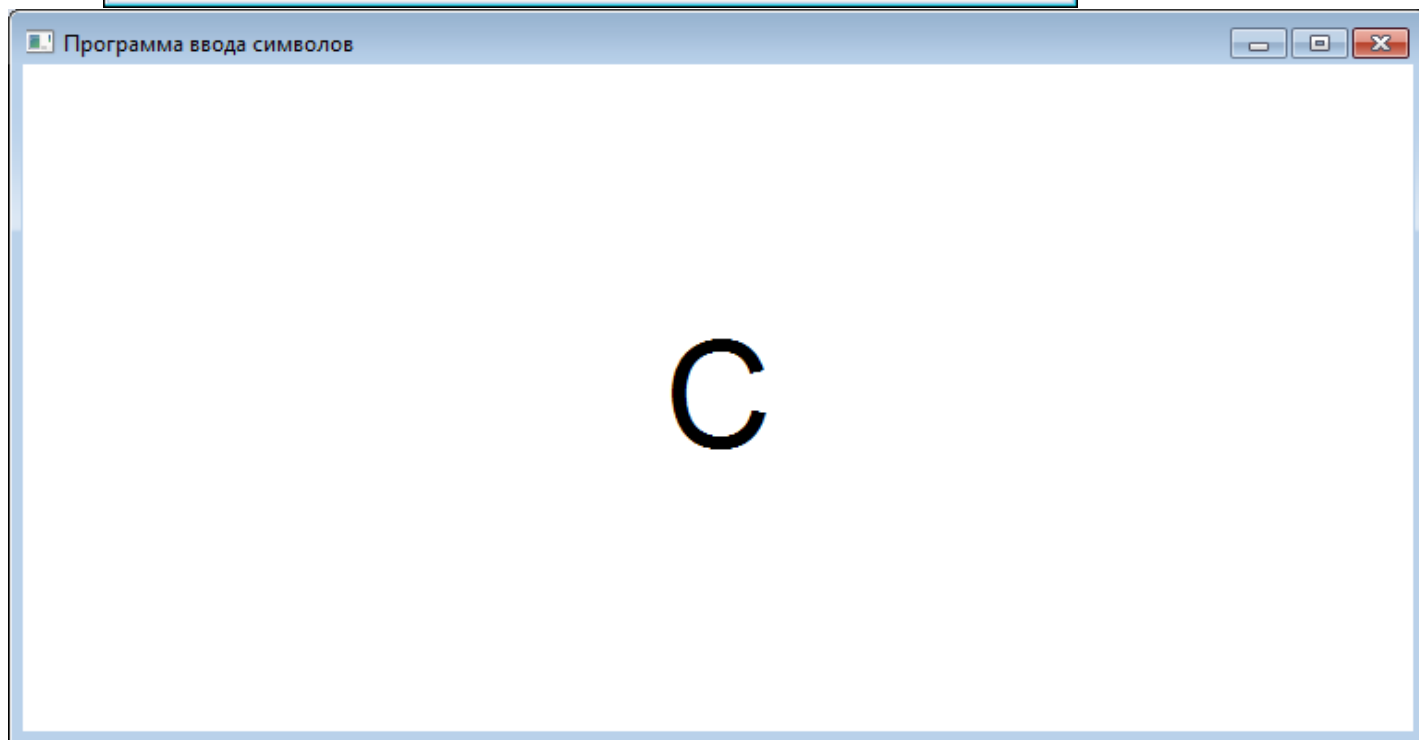
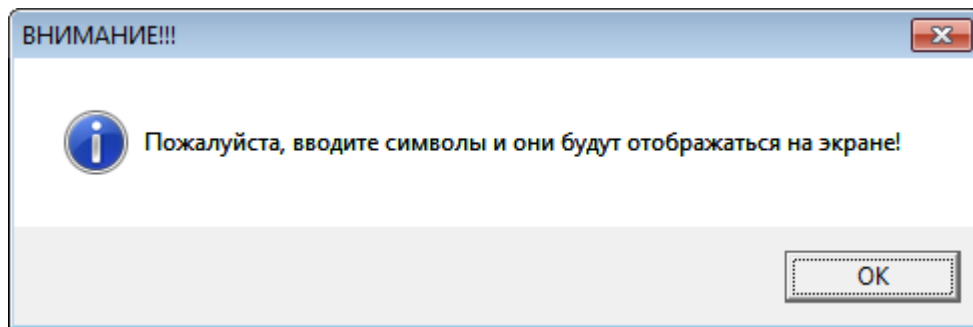
Наконец, обработчик `WM_PAINT`. С 75-й по 77-ю строки происходит вызов функций контекста устройства.

А с 78-й по 82-ю строки вызывается функция `CreateFont()`. С её помощи мы меняем шрифт, и размер буквы. Вот её описание:

```
1      HFONT CreateFont(int, // высота шрифта
2                      int, // ширина символов
3                      int, // угол наклона букв
4                      int, // угол наклона строки
5                      int, // толщина букв («жирность»)
6                      DWORD, // курсив
7                      DWORD, // подчеркивание
8                      DWORD, // перечеркивание
9                      DWORD, // набор символов
10                     DWORD, // точность вывода
11                     DWORD, // точность отсечения
12                     DWORD, // качество вывода
13                     DWORD, // шаг между буквами
14                     LPCTSTR // имя шрифта
15                     );
```

Шрифт выставлен Arial Bold – то есть Arial жирным. В строке 83 мы задаём контексту изменённые параметры шрифта. С 84-й по 85-ю строки ничего нового. Мы вводим на экран полученный идентификатор клавиши, преобразованный в тип `char`. В функции `DrawText()` его нужно преобразовать к типу `LPCWSTR`, что она и требует.

Результат работы программы при вводе буквы «с» такой:



Далее мы научимся обрабатывать дополнительные клавиши, переключать раскладку, создавать таймеры отсчёта.

Ввод и вывод текста. Буфер памяти, таймер отсчёта

В этой статье речь пойдёт о буфере и о таймере отсчёта.

Для начала убедитесь, что Вы разбираетесь в выделении памяти [с помощью функции alloc \(malloc\), free](#), [с помощью операторов new, delete](#) и понимаете функцию преобразованию [itoa\(\)](#).

Рассмотрим листинг, представленный ниже:

```
1      LRESULT CALLBACK WindowProcess(HWND hWindow, UINT uMessage,
2          WPARAM wParam, LPARAM lParam)
3      {
4          HDC hDeviceContext;
5          PAINTSTRUCT paintStruct;
6          RECT rectPlace;
7          HFONT hFont;
8
9          static PTCHAR text;
10         static int size = 0;
11
12         switch (uMessage)
13         {
14             case WM_CREATE:
15                 text=(PTCHAR)GlobalAlloc(GPTR,50000*sizeof(TCHAR));
16                 break;
```

```

17     case WM_PAINT:
18         hDeviceContext = BeginPaint(hWindow, &paintStruct);
19         GetClientRect(hWindow, &rectPlace);
20         SetTextColor(hDeviceContext, NULL);
21         hFont=CreateFont(50,0,0,0,0,0,0,0,
22             DEFAULT_CHARSET,
23             0,0,0,VARIABLE_PITCH,
24             "Arial Bold");
25         SelectObject(hDeviceContext,hFont);
26         if(wParameter != VK_RETURN)
27             DrawText(hDeviceContext,
28                 (LPCSTR)text,
29                 size, &rectPlace,
30                 DT_SINGLELINE|DT_CENTER|DT_VCENTER);
31         EndPaint(hWindow, &paintStruct);
32         break;
33     case WM_CHAR:
34         switch(wParameter)
35         {
36             case VK_RETURN:
37                 size=0;
38                 break;
39             default:
40                 text[size]=(char)wParameter;
41                 size++;
42                 break;
43         }
44         InvalidateRect(hWindow, NULL, TRUE);
45         break;
46     case WM_DESTROY:
47         PostQuitMessage(NULL);
48         GlobalFree((HGLOBAL)text);
49         break;
50     default:
51         return DefWindowProc(hWindow, uMessage, wParameter, lParameter);
52     }
53     return NULL;
54 }

```

Результат после ввода CppStudio.com будет таким:

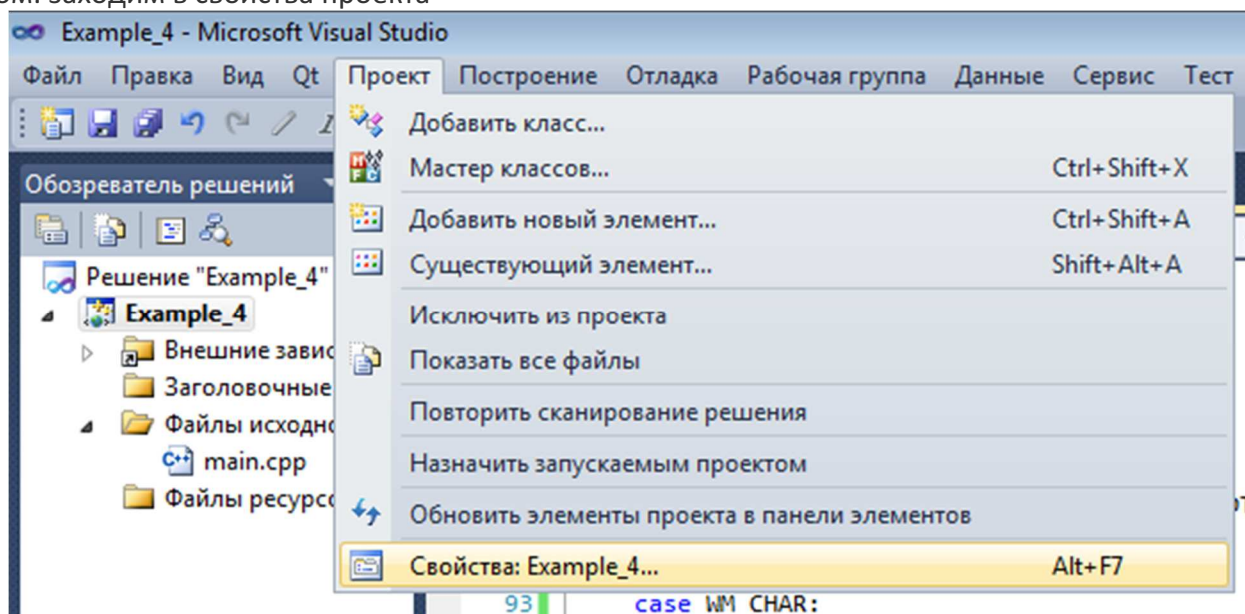


Код для функции `WinMain` не приводится, так как он не менялся с предыдущих уроков. Данная программа позволяет вводить текст по мере возможности и стирать его клавишей ENTER.

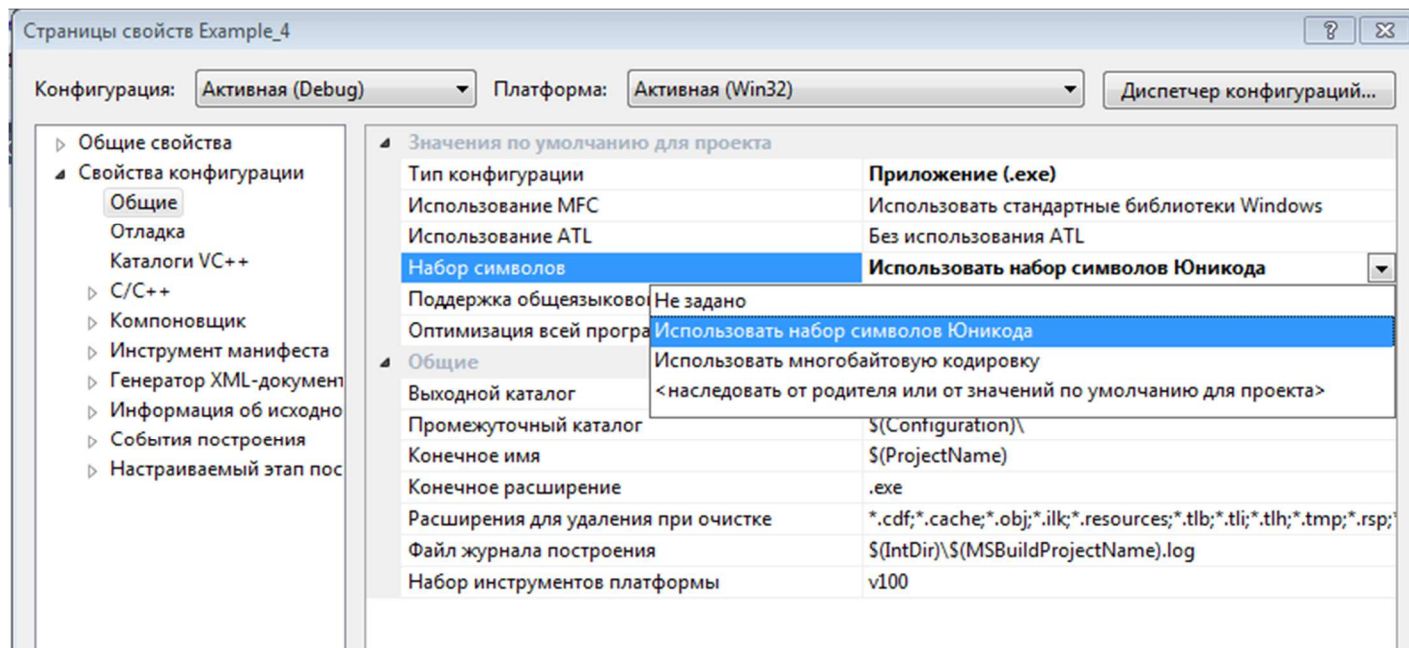
Наверное, Вы заметили, что в программе для считывания нажатой клавиши мы использовали message `WM_KEYDOWN`, а здесь что-то новое — `WM_CHAR`.

Отличительная особенность в том, что `WM_KEYDOWN` нужен для обработки специальных клавиш типа shift, ctrl и т.п., а `WM_CHAR` — отвечает за обработку символов. Обычно они работают вместе для комбинаций клавиш и вызывают друг друга. В этом Вы убедитесь, когда мы будем разбирать работу текстового редактора.

По умолчанию в MVS используется Юникод. Его отключение производится следующим образом: заходим в свойства проекта



и отключаем Юникод, нажав «Использовать многобайтовую кодировку».



После того, как Вы это сделаете, перед строками не нужно будет писать L, и представленный код не будет иметь ошибок. По этой причине в программе прошлого урока было не правильное отображение некоторых символов.

Теперь второе отличие: мы можем вводить текст (примерно до 50 кБ), который сохранится в буфере.

Конструкция создания, использования, удаления буфера выглядит так:

```
1 static TCHAR text;
2 text=(TCHAR)GlobalAlloc(GPTR,50000*sizeof(TCHAR));
3 if()
4 {
5     //....
6     // заполняем буфер информацией, используем её, где нам нужно
7     //.....
8     GlobalFree((HGLOBAL)text);
9 }
```

Она аналогична `alloc()` и `free()` и её смысл абсолютно такой же.

TCHAR — указатель на тип **TCHAR**.

GPTR — режим выделения памяти (в данном случае фиксированное)

HGLOBAL — тип возвращаемого значения функции `GlobalAlloc()`

Третье достоинство нашей программы — возможность стирать текст (**строки 36-38**).

В случае нажатия клавиши ENTER — мы делаем окно не действительным с помощью функции `InvalidateRect()` (**строка 44**), тем самым вызываем сообщение **WM_PAINT**. В нём прописано, что если параметр `wParameter` был равен **VK_RETURN** (то есть ENTER), то мы перерисовываем окно и не вызываем функцию `DrawText()`.

И, как и обещалось ранее, добавим в функцию таймер и некоторые другие особенности.

```
1 static int sec = 0;
2 TCHAR workTime[10];
3 //...
4 switch (wParameter)
5 {
6 case WM_CREATE:
7     SetTimer(hWindow, 1, 1000, NULL);
8     //...
9     SetForegroundWindow(hWindow);
10    break;
```

```

11     case WM_CHAR:
12         //...
13     case VK_BACK:
14         if(size!=0)
15             size--;
16         break;
17     case WM_TIMER:
18         sec++;
19         break;
20     case WM_DESTROY:
21         KillTimer(hWindow, 1);
22         _itoa(sec, workTime, 10); MessageBox(NULL, (LPCSTR)workTime, "Время работы программы", MB_OK);
23         //...
24         break;
25     }

```

Добавьте в соответствующие места программы выше представленный код.

Функция `SetTimer()` создаёт таймер. Первый параметр — дескриптор окна, третий — интервал срабатывания `WM_TIMER` в миллисекундах, второй — количество вызовов `WM_TIMER` в заданный интервал времени. Четвёртый параметр — нам не нужен.

В сообщении `WM_TIMER` мы увеличиваем `sec` на единицу.

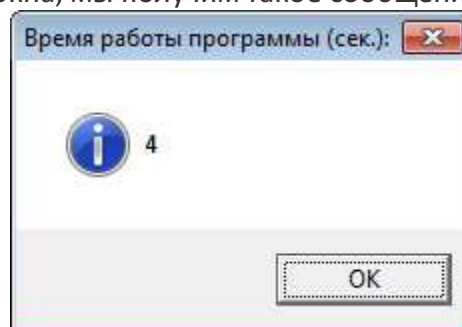
Функция `KillTimer()` — удаляет таймер. Параметры те же, что и в `SetTimer()`.

Чтобы отобразить время работы программы, в `WM_DESTROY` мы преобразовываем `int` в `TCHAR` массив при помощи функции `_itoa`, выводим строку на экран, удаляем таймер и освобождаем память, выделенную под буфер.

Функция `SetForegroundWindow(HWND)` — делает окно активным. Например, если Вы находитесь на сайте CppStudio.com и при этом идёт компиляция данной программы, это окно появится поверх браузера с фокусом для ввода.

Для возможности удаления одного или нескольких символов добавлен обработчик `VK_BACK` — нажатие клавиши `backspace`. Если `size` не равен нулю (то есть не был нажат `ENTER`), мы уменьшаем его на единицу и последний элемент при обновлении окна не отображается.

В результате, при закрытии окна, мы получим такое сообщение:



Вопросы.